

Documentação Técnica

Sistema de Emissão de Notas Fiscais
Teste Técnico Prático — Korp ERP

Bruno Cardoso Holanda

21 de agosto de 2026

Sumário

1	Objetivo e Escopo	2
2	Arquitetura Geral	2
2.1	Estrutura de pastas	2
3	Telas Desenvolvidas	3
4	Funcionalidades Implementadas	3
5	Frontend — Angular	3
5.1	Ciclos de vida do Angular utilizados	3
5.2	Uso da biblioteca RxJS	3
5.3	Outras bibliotecas utilizadas	4
5.4	Bibliotecas de componentes visuais	4
5.5	Gerenciamento de dependências (Golang)	4
6	Backend — C# / ASP.NET Core	5
6.1	Frameworks utilizados	5
6.2	Tratamento de erros e exceções	5
6.3	Uso de LINQ	5
7	Persistência de Dados	6
7.1	Modelo de dados	6
8	Cenário de Falha e Recuperação	6
9	Requisito Opcional: Tratamento de Concorrência	7
9.1	Mecanismo	7
9.2	Teste realizado	7

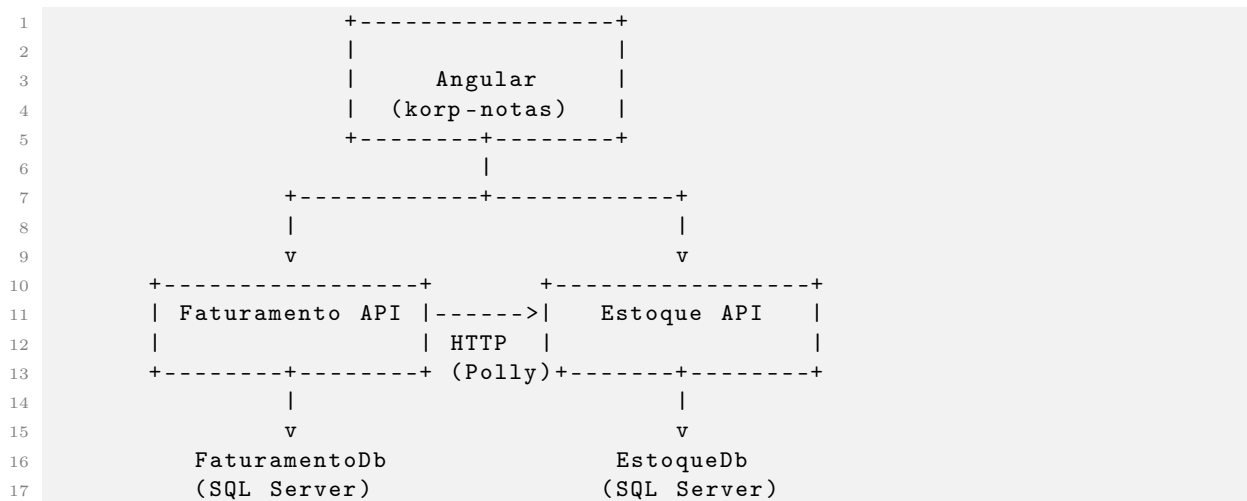
1 Objetivo e Escopo

O projeto consiste em uma aplicação de emissão de notas fiscais, desenvolvida em Angular no frontend, com backend composto por dois microsserviços independentes em C# / ASP.NET Core:

- **Estoque Service:** responsável pelo cadastro de produtos e controle de saldo em estoque;
- **Faturamento Service:** responsável pela criação, listagem e impressão (fechamento) de notas fiscais.

Os dois serviços possuem bancos de dados próprios (**EstoqueDb** e **FaturamentoDb**), reforçando a separação de responsabilidades entre os microsserviços. A comunicação entre os serviços ocorre via HTTP, com política de resiliência (retry) implementada com a biblioteca Polly.

2 Arquitetura Geral



Listing 1: Visão geral da arquitetura

2.1 Estrutura de pastas

```
1 Korp_Testes_BrunoCardosoHolanda/
2 |-- backend/
3 |   |-- EstoqueService/
4 |   |   |-- Controllers/
5 |   |   |-- Models/
6 |   |   '-- Data/
7 |   '-- FaturamentoService/
8 |       |-- Controllers/
9 |       |-- Models/
10 |       |-- Data/
11 |       '-- Services/
12 '-- frontend/
13     '-- korp-notas/
14         '-- src/app/
15             |-- core/services/
16             |-- models/
17             |-- produtos/
18             '-- notas/
```

3 Telas Desenvolvidas

- **Produtos:** listagem, cadastro, edição e exclusão de produtos (código, descrição e saldo);
- **Notas Fiscais:** listagem de notas com status (Aberta/Fechada);
- **Nova Nota:** criação de nota fiscal com seleção de múltiplos produtos e quantidades;
- **Detalhe da Nota:** visualização dos itens da nota e botão de impressão, com feedback visual de processamento, sucesso e falha.

4 Funcionalidades Implementadas

- CRUD completo de produtos;
- Criação de notas fiscais com múltiplos itens e numeração sequencial automática;
- Impressão de nota fiscal, que orquestra a baixa de estoque no microsserviço de Estoque e, em caso de sucesso em todos os itens, altera o status da nota para *Fechada*;
- Bloqueio de reimpressão de notas já fechadas;
- Cenário de falha entre microsserviços, com política de *retry* e mensagem de erro amigável ao usuário, mantendo a nota em estado *Aberta* para nova tentativa;
- Controle de concorrência otimista no saldo de produtos (detalhado na Seção 9).

5 Frontend — Angular

5.1 Ciclos de vida do Angular utilizados

O projeto utiliza componentes *standalone* (sem NgModules), estrutura padrão das versões recentes do Angular. Os seguintes ciclos de vida foram utilizados:

- **ngOnInit:** utilizado em todos os componentes de tela (`ProdutoList`, `ProdutoForm`, `NotaList`, `NotaForm`, `NotaDetalhe`) para disparar o carregamento inicial de dados via chamadas HTTP assim que o componente é inicializado – por exemplo, a listagem de produtos e notas, e o carregamento de um produto/nota específico ao entrar em modo de edição ou detalhe.

5.2 Uso da biblioteca RxJS

RxJS é utilizado de forma nativa através do `HttpClient` do Angular, que retorna `Observable` para toda chamada HTTP. O padrão utilizado em todos os componentes foi:

```
1 this.produtoService.listar().subscribe({
2   next: (produtos) => {
3     this.produtos = produtos;
4     this.carregando = false;
5     this.cdr.detectChanges();
6   },
7   error: () => {
```

```

8      this.snackBar.open('Erro ao carregar produtos.', 'Fechar', {
9          duration: 4000,
10         panelClass: ['snackbar-erro']
11     });
12     this.carregando = false;
13     this.cdr.detectChanges();
14 }
15 });

```

Listing 3: Padrão de subscribe com tratamento de sucesso e erro

Cada chamada assíncrona (GET, POST, PUT, DELETE) é tratada como um **Observable**, com os *callbacks* **next** e **error** separando o fluxo de sucesso do fluxo de falha – o que é a base do tratamento de erros do frontend com a nota fiscal (ex.: exibição de mensagem de falha na impressão sem quebrar a aplicação).

5.3 Outras bibliotecas utilizadas

- **Angular Reactive Forms** (ReactiveFormsModule, FormBuilder, FormGroup, Validators): utilizado nos formulários de produto e de itens da nota fiscal, com validações declarativas (campos obrigatórios, valores mínimos);
- **Angular Router**: navegação entre telas e roteamento por parâmetro (/produtos/:id/editar, /notas/:id);
- **RxJS** (nativo do HttpClient): tratado na seção anterior.

5.4 Bibliotecas de componentes visuais

Foi utilizado o **Angular Material** (Material Design 3), incluindo os módulos:

- **MatTableModule** – tabelas de listagem;
- **MatFormFieldModule** / **MatInputModule** / **MatSelectModule** – campos de formulário;
- **MatButtonModule** / **MatIconModule** – botões e ícones;
- **MatChipsModule** – indicador visual de status da nota (Aberta/Fechada);
- **MatSnackBarModule** – notificações de sucesso e erro;
- **MatProgressSpinnerModule** – indicadores de carregamento e processamento (ex.: durante a impressão da nota, enquanto o Faturamento aguarda resposta do Estoque);
- **MatToolbarModule** – barra de navegação principal.

O tema visual segue o sistema *Material 3*, com paleta de cores **azure/blue**, aplicado globalmente via `mat.theme()` no arquivo `styles.scss`.

5.5 Gerenciamento de dependências (Golang)

Não aplicável – o projeto não utiliza Golang em nenhuma camada. O gerenciamento de dependências do frontend é feito via **npm** (`package.json`), e do backend via **NuGet** (`.csproj`), conforme detalhado na Seção 6.

6 Backend — C# / ASP.NET Core

6.1 Frameworks utilizados

- **ASP.NET Core Web API** (.NET): base de ambos os microserviços, expondo endpoints RESTful via Controllers;
- **Entity Framework Core**: ORM utilizado para mapeamento objeto-relacional e persistência no SQL Server, incluindo o versionamento de esquema via *migrations*;
- **Polly**: biblioteca de resiliência utilizada no `HttpClient` do Faturamento Service para implementar política de *retry* com *backoff* exponencial nas chamadas ao Estoque Service;
- **Swashbuckle (Swagger)**: geração de documentação interativa dos endpoints (Swagger UI), utilizada durante todo o desenvolvimento para testes manuais dos microserviços.

6.2 Tratamento de erros e exceções

O tratamento de erros ocorre em duas camadas:

1. **Nível de negócio**: os Controllers retornam códigos HTTP apropriados para cada cenário – 404 Not Found (recurso inexistente), 400 Bad Request (regra de negócio violada, como saldo insuficiente ou tentativa de reimpressão de nota fechada), 409 Conflict (conflito de concorrência otimista), e 502 Bad Gateway (falha de comunicação entre microserviços);
2. **Nível de infraestrutura/resiliência**: o `EstoqueClientService`, no Faturamento, encapsula a chamada HTTP ao Estoque em um bloco `try/catch`, e a política do Polly intercepta erros transitórios (5xx, timeouts, falhas de rede) antes que cheguem à aplicação, tentando novamente até 3 vezes com espera progressiva (2s, 4s, 8s) antes de repassar a falha ao chamador.

```
1 static IAsyncPolicy<HttpResponseMessage> GetRetryPolicy()  
2 {  
3     return HttpPolicyExtensions  
4         .HandleTransientHttpError()  
5         .OrResult(msg => msg.StatusCode == HttpStatusCode.RequestTimeout  
6     )  
7         .WaitAndRetryAsync(3, retryAttempt =>  
8             TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)));  
9 }
```

Listing 4: Política de retry configurada com Polly

Quando todas as tentativas de retry se esgotam, o endpoint de impressão da nota (POST `/api/notas/{id}/imprimir`) retorna uma mensagem de erro compreensível ao usuário, e a nota permanece com status *Aberta*, permitindo nova tentativa quando o serviço de Estoque estiver disponível novamente.

6.3 Uso de LINQ

LINQ é utilizado extensivamente nas consultas ao banco de dados via Entity Framework Core, entre outros pontos:

```
1 // Calculo do proximo numero sequencial de nota fiscal  
2 var ultimoNumero = await _context.NotasFiscais  
3     .MaxAsync(n => (int?)n.Numero) ?? 0;  
4
```

```

5 // Carregamento de nota com seus itens relacionados (eager loading)
6 var nota = await _context.NotasFiscais
7     .Include(n => n.Itens)
8     .FirstOrDefaultAsync(n => n.Id == id);
9
10 // Verificacao de existencia apos falha de concorrencia
11 if (!await _context.Produtos.AnyAsync(p => p.Id == id))
12     return NotFound();
13
14 // Projecao de itens de uma requisicao para entidades de dominio
15 Itens = request.Itens.Select(i => new ItemNotaFiscal
16 {
17     ProdutoId = i.ProdutoId,
18     Quantidade = i.Quantidade
19 }).ToList()

```

Listing 5: Exemplos de uso de LINQ nos Controllers

7 Persistência de Dados

Ambos os microsserviços utilizam **SQL Server Express**, com um banco de dados dedicado por serviço (**EstoqueDb** e **FaturamentoDb**), reforçando o isolamento entre os *bounded contexts* de Estoque e Faturamento. O versionamento do esquema é feito via *migrations* do Entity Framework Core (`dotnet ef migrations add` / `dotnet ef database update`).

7.1 Modelo de dados

Estoque Service

- Produto: Id, Codigo, Descricao, Saldo, RowVersion (controle de concorrência).

Faturamento Service

- NotaFiscal: Id, Numero, Status (*Aberta/Fechada*), DataCriacao;
- ItemNotaFiscal: Id, NotaFiscalId, ProdutoId, Quantidade.

8 Cenário de Falha e Recuperação

Este é o requisito obrigatório central do desafio. O fluxo de impressão de uma nota fiscal foi projetado para lidar explicitamente com a indisponibilidade do microsserviço de Estoque:

1. O usuário clica em *Imprimir* na tela de detalhe da nota;
2. O Faturamento Service chama o endpoint de baixa de estoque para cada item da nota;
3. Caso o Estoque Service esteja indisponível, o Polly tenta novamente até 3 vezes, com espera progressiva;
4. Se todas as tentativas falharem, o Faturamento retorna erro ao Angular, que exibe uma mensagem clara ao usuário (*"Não foi possível processar a nota. Serviço de Estoque indisponível. Tente novamente."*), sem alterar o status da nota;
5. Quando o serviço de Estoque volta a ficar disponível, o usuário pode clicar em *Imprimir* novamente, e a operação é concluída com sucesso, demonstrando a recuperação do sistema.

Esse fluxo foi validado manualmente, encerrando o processo do Estoque Service durante a impressão de uma nota e confirmando o comportamento descrito acima, tanto na resposta da API quanto na interface do usuário.

9 Requisito Opcional: Tratamento de Concorrência

Foi implementado e validado o tratamento de concorrência otimista para o cenário descrito na especificação: um produto com saldo igual a 1 sendo utilizado simultaneamente por duas notas fiscais.

9.1 Mecanismo

A entidade `Produto` possui uma coluna do tipo `rowversion` (`RowVersion`), mapeada como *currency token* no Entity Framework Core. Toda escrita no banco de dados sobre um registro de produto inclui automaticamente uma cláusula de comparação com o valor do `RowVersion` lido no momento da consulta. Caso duas requisições concorrentes leiam o mesmo estado do produto e tentem gravar alterações, apenas a primeira escrita é aceita pelo SQL Server – a segunda falha com uma `DbUpdateConcurrencyException`, que é capturada no endpoint de baixa de estoque e traduzida em uma resposta 409 `Conflict`.

```
1 try
2 {
3     await _context.SaveChangesAsync();
4 }
5 catch (DbUpdateConcurrencyException)
6 {
7     return Conflict(new { erro = "Conflito de concorrencia. Tente
8     novamente." });
9 }
```

Listing 6: Tratamento de concorrência no endpoint de baixa de estoque

9.2 Teste realizado

O cenário foi validado da seguinte forma:

1. Um produto foi ajustado para `saldo = 1`;
2. Duas notas fiscais distintas foram criadas, cada uma referenciando esse produto com quantidade 1;
3. As duas requisições de impressão (`POST /api/notas/{id}/imprimir`) foram disparadas simultaneamente, utilizando processamento em paralelo (*PowerShell jobs*), simulando concorrência real;
4. **Resultado:** apenas uma das notas foi processada com sucesso (status alterado para *Fechada*, saldo do produto reduzido para 0); a segunda nota falhou com erro de conflito e permaneceu com status *Aberta*, sem que o saldo do produto se tornasse negativo ou fosse debitado duas vezes.

Esse teste confirma que o sistema garante consistência do saldo de estoque mesmo sob concorrência real entre múltiplas notas fiscais.